



CMC UNIVERSITY

Aspire to Inspire the Digital World

ENTITY FRAMEWORK

Ngô Việt Anh

Khoa CNTT&TT, Trường Đại học CMC

Hà Nội, tháng 12 năm 2025

- **Entity Framework (EF)** là một công cụ ORM (Object-Relational Mapping) được sử dụng trong .NET Core để làm việc với cơ sở dữ liệu theo cách đối tượng hóa. Với **Entity Framework Core (EF Core)**, bạn có thể dễ dàng thao tác với dữ liệu mà không cần viết nhiều mã SQL.

- **Là phiên bản nhẹ hơn, hiện đại hơn của EF:** EF Core được thiết kế để hoạt động tốt trên nhiều nền tảng, bao gồm Windows, Linux, và macOS.
- **Hỗ trợ đa cơ sở dữ liệu:** EF Core hỗ trợ các loại cơ sở dữ liệu như SQL Server, SQLite, MySQL, PostgreSQL, Oracle, Cosmos DB, v.v.
- **Mô hình hóa dữ liệu:** EF Core cho phép ánh xạ các đối tượng C# (classes) sang các bảng trong cơ sở dữ liệu.

EF Core sử dụng một **DbContext** để giao tiếp với cơ sở dữ liệu. Nó bao gồm các bước cơ bản:

- **Khai báo các mô hình (Model Classes):**
 - Mỗi class tương ứng với một bảng trong cơ sở dữ liệu.
- **Cấu hình ánh xạ bằng DbContext:**
 - Sử dụng DbContext để quản lý các mô hình và ánh xạ chúng tới cơ sở dữ liệu.
- **Chạy migrations (di chuyển dữ liệu):**
 - Tự động tạo và quản lý cấu trúc cơ sở dữ liệu dựa trên mô hình.

- **Code First:** Cho phép bạn định nghĩa các mô hình và ánh xạ chúng trực tiếp trong code. Sau đó EF Core sẽ tự động tạo cơ sở dữ liệu dựa trên các mô hình này.
- **Database First:** Sử dụng cơ sở dữ liệu hiện có và tạo các mô hình từ cấu trúc đó.
- **Query LINQ:** Hỗ trợ ngôn ngữ truy vấn LINQ (Language Integrated Query) để truy xuất dữ liệu theo cách mạnh mẽ và linh hoạt.
- **Change Tracking:** Theo dõi sự thay đổi của các thực thể (entities) và tự động cập nhật chúng vào cơ sở dữ liệu.
- **Lazy Loading:** Trì hoãn tải dữ liệu liên quan cho đến khi cần thiết.
- **Eager Loading:** Tải trước các dữ liệu liên quan trong một truy vấn duy nhất.

- Cài đặt thư viện qua NuGet:
 - Microsoft.EntityFrameworkCore
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools

1. Thêm thư viện
2. Tạo mô hình dữ liệu
3. Tạo DbContext
4. Migration và Cập nhật cơ sở dữ liệu
5. Sử dụng trong ứng dụng

- Add-Migration: Tạo 1 phiên bản của db trên code
- Update-Database: Cập nhật phiên bản mới nhất lên database
- Drop-Database: Xóa DB
- Get-Migration: Lấy danh sách các phiên bản migration hiện có
- Remove-Migration: Xóa phiên bản mới nhất hoặc truyền tên phiên bản cần xóa
- Script-Migration: Tạo ra các câu lệnh SQL phục vụ cho việc update DB


```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }

    // Lazy Loading: Navigation Property phải là virtual
    public virtual List<Order> Orders { get; set; }
}

public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }
    public virtual Customer Customer { get; set; }
    public decimal TotalPrice { get; set; }
}
```

Lazy Loading là kỹ thuật tải dữ liệu liên quan một cách lười biếng – tức là dữ liệu chỉ được lấy từ database khi nó thực sự được sử dụng.

- Cách hoạt động:
 - Khi truy vấn một đối tượng chính (Customer), dữ liệu liên quan (Orders) không được load ngay lập tức.
 - Khi truy cập customer.Orders, Entity Framework mới gửi truy vấn để lấy dữ liệu từ database

```
using (var context = new AppDbContext())
{
    var customers = context.Customers.ToList(); // Chỉ lấy danh sách Customers

    foreach (var customer in customers)
    {
        var orders = customer.Orders; // Truy vấn chỉ thực hiện khi truy cập Orders
    }
}
```

- **Eager Loading** là kỹ thuật **tải trước** các dữ liệu liên quan ngay khi truy vấn được thực thi. Điều này giúp **giảm số lượng truy vấn** gửi đến database, tránh tình trạng **N+1 Query Problem** (truy vấn lồng nhau quá nhiều).
- **Lợi ích:**
 - **Giảm số lần truy vấn** (so với Lazy Loading).
 - **Tăng tốc độ xử lý** với dữ liệu quan hệ phức tạp.
 - **Tránh lỗi dữ liệu bị null** khi truy xuất từ các bảng liên quan

```
var customers = context.Customers  
    .Include(c => c.Orders) // Load luôn danh sách đơn hàng  
    .ToList();
```

```
var customers = context.Customers  
    .Include(c => c.Orders)  
        .ThenInclude(o => o.OrderDetails) // Load luôn chi tiết đơn hàng  
    .ToList();
```

Tiêu chí	Lazy Loading	Eager Loading
Cách hoạt động	Chỉ tải dữ liệu khi truy cập property	Tải toàn bộ dữ liệu ngay khi truy vấn
Số lượng truy vấn	Nhiều truy vấn nhỏ (có thể gây N+1 Query Problem)	1 truy vấn lớn (tốt hơn khi cần toàn bộ dữ liệu)
Hiệu suất	Tiết kiệm tài nguyên nếu không phải lúc nào cũng cần dữ liệu liên quan	Tốt hơn khi cần tất cả dữ liệu liên quan ngay từ đầu
Cách sử dụng	Dùng khi không cần dữ liệu liên quan ngay lập tức	Dùng khi biết chắc sẽ cần dữ liệu liên quan

- **Data Annotation** là cách sử dụng thuộc tính (attributes) của C# để định nghĩa ràng buộc trên các model trong Entity Framework Core. Nó giúp giảm thiểu lỗi và kiểm soát dữ liệu tốt hơn mà không cần viết Fluent API.

- [Key] – Định nghĩa Khóa Chính (Primary Key)

```
public class Customer
{
    [Key]
    public int CustomerId { get; set; } // Khóa chính
    public string Name { get; set; }
}
```

- [Required] – Bắt buộc nhập (Not Null)

```
public class Customer
{
    public int Id { get; set; }

    [Required]
    public string Name { get; set; }
}
```

- [MaxLength] và [StringLength] – Giới hạn độ dài chuỗi

```
public class Customer
{
    public int Id { get; set; }

    [StringLength(50)] // Độ dài tối đa 50 ký tự
    public string Name { get; set; }

    [MaxLength(100)] // Giới hạn 100 ký tự (Không tạo ràng buộc DB)
    public string Address { get; set; }
}
```

Thuộc tính	Ảnh hưởng Database?	Ảnh hưởng Validation?
[StringLength(50)]	✓ (Tạo cột VARCHAR(50))	✓ (Xác thực dữ liệu nhập)
[MaxLength(100)]	✗ (Không tạo ràng buộc DB)	✓ (Xác thực dữ liệu nhập)

- [Column] – Tùy chỉnh tên cột

```
public class Customer
{
    public int Id { get; set; }

    [Column("CustomerFullName")]
    public string Name { get; set; }
}
```

- [Table] – Đổi tên bảng

```
[Table("KhachHang")]  
public class Customer  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
}
```

- [ForeignKey] – Định nghĩa Khóa Ngoại

```
public int Id { get; set; }

[ForeignKey("Customer")] // Khóa ngoại tham chiếu tới bảng Customer
public int CustomerId { get; set; }

public Customer Customer { get; set; }
```

- [NotMapped] – Loại bỏ cột khỏi Database

```
public int Id { get; set; }  
public string Name { get; set; }  
  
[NotMapped]  
public string FullName { get; set; } // Không Lưu vào DB
```

- [ConcurrentCheck] – Kiểm tra xung đột dữ liệu

```
public class Product
{
    public int Id { get; set; }

    [ConcurrentCheck]
    public int Stock { get; set; }
}
```

- [Timestamp] – Kiểm soát phiên bản bản ghi

```
public class Product
{
    public int Id { get; set; }

    [Timestamp]
    public byte[] RowVersion { get; set; }
}
```




CMC UNIVERSITY

Aspire to Inspire the Digital World



THANK YOU